DAY 10 · MOCK EXAM 2

Mock Exam #2

A second timed rehearsal, then the exam-day playbook.

120 min + strategy / day10_mock_exam_2 / Exam strategy

INSIDE THIS BOOKLET

Key-value store | Strategy | Cheat sheet

How to use this booklet

Each day is a working session, not a reading assignment. This booklet is the typeset version of the day's `README.md`: keep the folder open, type the examples into a Python REPL, and run the tests as you go. The *Exam lens* and end-of-day review are study aids added to the original material.

Read & type
Type every example. Predict the output first.

Practice
Implement or debug one test at a time.

Test
Run the suite. Read the first failing line.

Log
Record each bug in `bug_journal.md`.

You will need: Python 3.10+ · a terminal · `day10_mock_exam_2/` · the course runner `python run.py 10`

In this session

Part 1 — Mock exam: broken transactional key-value store 3

Part 2 — Exam-day strategy (read the evening before) 6

End-of-day checklist 7

> ◎ **The brief**
>
> A second 120-minute rehearsal – a transactional key-value store – followed by the exam-day playbook.

Part 1 — Mock exam: broken transactional key-value store

Same rules as Day 9: 120-minute timer, no AI/web/solutions, only edit `exam/kvstore.py`.

```
cp exam/kvstore.py exam/kvstore.original.py     # pristine copy for a retry
cd exam
python -m unittest 2>&1 | tail -3
```

Read `exam/SPEC.md` first. 35 tests. Afterwards: `python run.py 10 --solutions`, then `solution/BUGS.md`. This exam is deliberately different from yesterday's. Day 9 was an algorithm working on bytes and lists. Today's program is a **class whose state changes over time** (transactions), plus a **binary file format** and a small **command parser**. Mixing several topics in one program like this is typical of the real assessment.

Background you need (allowed before starting the timer)

What a key-value store is A **key-value store** is essentially a dict wrapped in a class, with methods to `set` a key, `get` it, `delete` it, `count` how many keys hold a given value, and list the `keys` that start with a prefix. Real examples are databases like Redis. On its own, that part is Day 3 material. Three features make it interesting.

Feature 1: nested transactions A **transaction** is a group of changes that you can either keep or throw away *as a whole*. You call `begin()` to start one, make some changes, and then either `commit()` to make them permanent, or `rollback()` to undo all of them as if they never happened. Banks use transactions so that a transfer either fully happens or doesn't happen at all. **Nested** means you can call `begin()` again while a transaction is already open, opening a transaction inside a transaction. In this spec:

- `rollback()` undoes only the **innermost** open transaction, the most recent `begin`;
- `commit()` makes **all** open transactions permanent and closes them all.

How can that be implemented? The standard idea is a **stack of layers**:

- The committed data is a normal dict.
- Each `begin()` adds a new, empty dict on top of the stack. It will record only the changes made inside that transaction.
- A `set` writes into the **top** layer only.
- A `get` looks for the key **starting from the top layer and going down**, and returns the first value it finds. So the most recent change wins.
- `rollback()` throws away the top layer. Everything it recorded disappears in one go.
- `commit()` applies all the layers to the committed data, **from the bottom up**, so newer changes overwrite older ones. Then the stack is emptied.

Here's the example from the spec, traced step by step (☒ is explained just below):

```
committed: {a: 10}          layers: []
begin                       layers: [{}]
set a 20                    layers: [{a: 20}]          get a -> 20   (found in the top layer)
begin                       layers: [{a: 20}, {}]
delete a                    layers: [{a: 20}, {a: ⌧}]   get a -> None (the top layer says
    "deleted")
rollback                    layers: [{a: 20}]           get a -> 20
rollback                    layers: []                 get a -> 10   (only the committed data
    is left)
```

(In `kvstore.py`, the committed data is the attribute holding the main dict, and `_layers` is the list of open transactions, innermost last.)

Why a delete needs a "tombstone" Look at the `delete a` step above. Inside a transaction, deleting a key can't simply remove it from the top layer, because the key might still exist in a lower layer (here `{a: 20}`) or in the committed data (`{a: 10}`). A `get` would then fall through to the lower layer and find the old value, as if the delete never happened. So instead, a delete inside a transaction writes a special marker into the top layer, which means "this key is deleted, as far as this layer and everything above it is concerned". This marker is called a **tombstone**. In Python it's usually a unique object created just for this purpose, such as `_DELETED = object()`, which can never be confused with a real value.

- A `get` that finds a tombstone stops searching and reports the key as absent.
- A `commit` that finds a tombstone removes the key from the committed data.
- `count` and `keys` must use the same rules. Each of them must look at the **visible** state, which is what `get` would see, not at the committed dict alone. A query that reads the committed dict directly is a typical planted bug.

Feature 2: the binary snapshot format `serialize()` turns the visible contents of the store into bytes, a "snapshot" that could be saved to a file. `deserialize(data)` rebuilds a store from those bytes. The format, from the spec:

```
b"KV1"            3 bytes: a fixed "magic" signature identifying the format
N                 4 bytes, big-endian: number of entries
for each entry (sorted by the UTF-8 bytes of the key):
    key length    2 bytes, big-endian, counted in BYTES
    key           the key, encoded as UTF-8
    value length  4 bytes, big-endian, counted in BYTES
    value         the value, encoded as UTF-8
```

All of this is Day 5 and Day 8 material:

- The **magic** prefix is a fixed signature at the start of the file, so the reader can immediately reject data that isn't in this format.
- Lengths and counts are fixed-width integers written with `to_bytes(n, "big")` and read with `int.from_bytes(..., "big")`. Mixing up the width (2 vs 4 bytes) or the byte order breaks the round trip.
- Lengths count **bytes**: `len(key.encode("utf-8"))`, not `len(key)`. This is the D4 bug from the catalog, and it only shows up with non-ASCII keys like `"é"`.
- The reader is a cursor-based parser (Day 8, section 2). It must check that enough bytes remain *before every read*, reject bytes left over at the end, reject invalid UTF-8, and check that the keys arrive in strictly increasing order (which also rules out duplicates).

Feature 3: the command language `execute("SET name Ana Maria")` takes one line of text, works out which command it is, and calls the right method. That's Day 2 material, and the spec is precise about every detail:

- The command word is **case-insensitive**: `set`, `SET` and `Set` are all the same. That suggests `.upper()` on the first word only, since keys are case-sensitive.
- Words are separated by **any** whitespace, which means `split()` with no argument (Day 2), not `split(" ")`.
- For `SET`, the value is "everything after the key", and it may contain spaces. Splitting into all words would lose those spaces. A **limited split**, `line.split(None, 2)`, splits at most twice and leaves the rest of the line, internal spaces included, as the third piece. (`None` as the separator means "any whitespace", just like `split()` with no argument.)
- The output table in the spec says exactly which string to return in each case, including `NULL`, `ERROR` and `NO TRANSACTION`. Check the test for the exact spelling.

Python you'll meet in `kvstore.py` As with yesterday's exam, make sure you can read these lines at a glance before you start the timer. **A sentinel compared with `is`.** `_DELETED = object()` creates a brand-new, featureless object. Nothing else in the program can ever *be* that object, so `value is _DELETED` (Day 4, section 10) is a safe test for "this is a tombstone". Using `==`, or a value like `None` or `""` as the marker, could clash with real data. **Searching the layers from the top.** `for layer in reversed(self._layers):` visits the innermost (most recent) transaction first, then the ones below it (`reversed`, Day 1 section 8). The first layer that mentions the key decides the answer, which is why the order matters. A plain `for layer in self._layers:` would let an *older* layer win. **The top layer.** `self._layers[-1]` is the innermost open transaction (the last item of the list), and `self._layers[-1][key] = value` writes into that layer's dict. `if self._layers:` checks whether any transaction is open at all: an empty list is falsy (Day 1, section 3). **Removing a key that might not be there.** `view.pop(key, None)` removes `key` if it's present and does nothing if it isn't. Without the second argument, `pop` raises `KeyError` for a missing key (Day 3, section 1). **Fixed-size integers in bytes.** `len(items).to_bytes(4, "big")` writes a length as exactly 4 bytes, and `int.from_bytes(chunk, "big")` reads it back (Day 5, Day 8). The spec says how many bytes each length field uses; check that every `to_bytes` and every `take` agrees with it. **Sorting pairs of bytes.** `sorted((k.encode("utf-8"), v.encode("utf-8")) for k, v in ...)` builds `(key_bytes, value_bytes)` pairs and sorts them by key bytes first (tuples compare item by item). Bytes compare byte by byte, like strings compare character by character. **A nested function that moves the cursor: `nonlocal`.** `deserialize` reads the snapshot with a cursor `pos` (Day 8, section 2) and a small helper defined inside it:

```
pos = 3

def take(n):
    nonlocal pos
    if pos + n > len(data):
        raise ValueError("truncated snapshot")
    chunk = data[pos:pos + n]
    pos += n
    return chunk
```

A nested function can *read* the outer function's variables freely. But as soon as it *assigns* to one (`pos += n`), Python treats the name as a new local variable of the inner function, unless you declare `nonlocal pos`, which means "the `pos` I'm assigning to is the one in the enclosing function". Forget it, and the first `pos + n` fails with `UnboundLocalError`, because the inner `pos` doesn't have a value yet. With it, every call to `take` hands back the next `n` bytes and moves the shared cursor forward, so the caller can simply write `take(2)`, `take(4)`, … **`@classmethod` and `cls()`.** `deserialize` is called on the class, `KVStore.deserialize(data)`, and builds the new store with `cls()` (Day 4, section 4). **Repeating without a counter.** `for _ in range(count):` runs the body `count` times; the `_` says the loop number itself isn't needed (Day 1, section 8).

What calls what The data methods and the command parser all depend on the "find the visible value" logic, and `serialize` depends on `keys` and `get`. Following the Day 6 triage rule, fix the transaction and lookup tests first: the others are built on top of them.

After the timer

> **Compare the two runs**
>
> Compare your Day 9 and Day 10 results: did you get faster? Which kind of bug cost you the most time? That's what to drill in the remaining day.

Then read `solution/BUGS.md` and do the same review as yesterday: for every bug you missed, which test pointed at it, and what would have led you there faster?

Part 2 — Exam-day strategy (read the evening before)

Before

- Test CodeSignal on the **same machine and network** you'll use. Big monitor. Charger plugged in. Close Slack/mail/notifications.
- Have a sheet of paper and a pen. You'll do hand computations (bytes, small examples).
- Sleep. Seriously — debugging speed drops sharply when you're tired.

The 120 minutes

The schedule below is the Day 6 workflow with time limits attached. The limits are there because, under pressure, it's very easy to spend forty minutes on the first failure you happen to look at.

Time	What
0–10	Read the whole spec. Skim the code top to bottom (what calls what). Skim test names. Run all tests, write down the baseline score.
10–20	Triage: list failing tests grouped by function. Identify the low-level helpers.
20–100	Fix loop: simplest failing test of the lowest-level function → hand-compute expected → find divergence → minimal fix → **re-run the full suite**.
100–110	Pick up the tests you skipped earlier.
110–120	Remove debug prints and `breakpoint()`s. Final full run. Don't start risky refactors now.

Rules of thumb

Each of these rules comes from a common way of losing points:

1. **The tests are the spec.** When unsure about formats (spaces, order, exception type), read the test.
2. **Minimal diffs.** Planted bugs are 1-line changes. If you're rewriting a whole function, stop and look again. (Exception: if a function is hopeless and small, rewriting it from the spec can be faster. Your call.)
3. **One bug at a time, re-run every time.** Several bugs can hide each other. A fix that makes the score *drop* is a signal (either the fix is wrong, or it uncovered another bug — check which tests changed).
4. **Never let a test hang the suite.** If the run hangs, Ctrl-C and run classes one by one; look for a `while` loop whose index doesn't advance, or something growing without bound.
5. **Time-box.** 10–15 minutes without progress → note it and move on.
6. **Score is all that counts.** No points for style. Don't refactor, don't add type hints, don't clean up.

7. **Check what you can in the REPL** (`python3`, `help(str.partition)`, `"é".encode()`) instead of guessing.

A word on rule 3, because "bugs hiding each other" is confusing the first time it happens. Suppose both `get_stats` and `merge` are broken, and every training test uses both. You fix `get_stats` correctly, and the score... doesn't move, because `merge` still breaks those same tests. The temptation is to undo your fix, thinking it didn't work. Don't. If the change matches the spec, keep it, and check that the *helper's own* tests (`TestGetStats`) now pass. The bigger tests will turn green once the other bug is fixed too.

Last-minute cheat sheet

One line per trap from the course. If a line doesn't make immediate sense, go back to the day it came from and reread that section.

```
range(a, b) excludes b          s[-0:] == s                 sort() -> None
split() vs split(" ")           strip("ab") = char set     removeprefix() exists
max(d, key=d.get): first on ties    sorted(key=(-count, name)) for desc/asc
b[0] is int, b[0:1] is bytes    join bytes THEN decode      len(s) != len(s.encode())
0xxxxxxx | 110xxxxx 10xxxxxx | 1110xxxx 10.. 10.. | 11110xxx 10.. 10.. 10..
cont: b & 0xC0 == 0x80   payload masks 1F/0F/07 lead, 3F continuation, shift 6
mutable default arg / class attr / alias vs copy / __eq__ without __hash__
enumerate(x, start=1)    int.to_bytes(n, "big")    int.from_bytes(b, "big")
python -m unittest -v 2>&1 | grep -E "FAIL|ERROR"     python -m unittest mod.Class.test
breakpoint()   n s c r p ll w u d   b 42, i == 3   q
```

Good luck.

End-of-day checklist

Tick every box

- ☐ I completed the second 120-minute run and recorded my score.
- ☐ I compared Day 9 and Day 10: time per bug, missed patterns, score.
- ☐ I read the exam-day strategy and the last-minute cheat sheet.

Score: _____ / 35 **Bugs found:** _____

Exam lens

- Protect the score: minimal diffs, no refactors, no style points.
- Keep a 10-minute buffer to remove debug code and run the full suite.
- Use the cheat sheet for formats, not for logic you can derive.